INTRODUCTION

CHAPTER 1:

INTRODUCTION

1.1 ABOUT THE PROJECT

In today's digital world, smartphones have become an essential part of daily life. Users continuously receive notifications from messaging applications, social media platforms, banking services, email clients, and system applications. These notifications improve communication efficiency and ensure users stay updated in real time. However, they often display sensitive information directly on the lock screen or notification panel, creating significant privacy concerns.

Many notifications reveal personal conversations, financial alerts, One-Time Passwords (OTPs), and confidential information without requiring device authentication. In public environments, this can lead to unintended exposure of private data.

To address this issue, the project **Mirage Notify – A System for Real-Time Editing of Incoming Android Notifications** is developed. Mirage Notify is an Android application that intercepts system notifications and modifies their visible content based on user-defined rules. The application is developed using Java and XML in Android Studio and targets Android version 12 (API 31) and above.

The system is built using Android's NotificationListenerService, which securely monitors incoming notifications. When a notification matches predefined criteria, the system cancels the original notification and reposts a modified version while preserving its original functionality. This ensures that user interaction with the notification still opens the source application correctly.

The project demonstrates a non-invasive, privacy-enhancing solution that works without root access and without modifying the source applications.

1.2 BACKGROUND OF THE STUDY

Android provides users with basic notification privacy controls, such as hiding sensitive content on the lock screen. However, these controls apply globally to all applications. There is no built-in mechanism that allows users to selectively modify notification content based on sender, application, or message content.

With the rapid growth of digital communication, there is an increasing demand for customizable privacy controls. Users require a system that allows fine-grained control over how notifications appear on their devices.

Mirage Notify addresses this gap by providing:

1. Real-time notification interception
2. Rule-based modification
3. Selective content masking
4. Local logging of notifications

This enhances user privacy while maintaining usability.

1.3 PROBLEM STATEMENT

Mobile notifications frequently expose sensitive information on the device screen without requiring user authentication. This creates privacy risks in public or shared environments.

The major problems identified are:

1. Lack of selective notification masking
2. No rule-based modification system
3. Exposure of confidential messages on lock screen
4. Limited customization in the existing Android notification system

Therefore, there is a need for a system that provides granular control over notification content without affecting the original application's functionality.

1.4 OBJECTIVE OF THE PROJECT

The primary objectives of the Mirage Notify system are:

1. To intercept incoming notifications using NotificationListenerService.
2. To design a rule-based filtering mechanism for notifications.
3. To modify notifications using a cancel-and-re post methodology.
4. To preserve the original Pending-intent to maintain functionality.
5. To store notification logs using Room database.
6. To develop a minimal and efficient user interface using Bottom Navigation.
7. To ensure compatibility with Android 12 and above.

1.5 SCOPE OF THE PROJECT

The scope of Mirage Notify includes:

- Intercepting notifications from all installed applications.
- Applying user-defined rules for content modification.
- Masking sender names or hiding message content.
- Storing notification logs locally.
- Operating without root access.

The project does not modify the internal behavior of source applications. It only controls how notifications are displayed on the user's device.

SYSTEM ANALYSIS

CHAPTER 2:

SYSTEM ANALYSIS

2.1 INTRODUCTION

System study is the process of examining the current system to identify its limitations and determining the requirements for an improved solution. This chapter presents the initial investigation carried out for the Mirage Notify system, analysis of the existing Android notification mechanism, and details of the proposed system.

The main focus of this study is to analyze privacy issues related to Android notifications and to design a practical solution that enhances user control over notification content.

2.2 INITIAL INVESTIGATION

The initial investigation phase involved studying how Android handles notifications and understanding the privacy limitations faced by users.

During this phase, the following activities were performed:

- Study of Android Notification Framework
- Analysis of NotificationListenerService API
- Examination of lock screen notification behavior
- Understanding PendingIntent mechanism
- Evaluation of Android privacy settings

Observations:

1. Notifications display full content on the lock screen.
2. Android provides only global content hiding options.
3. No selective sender-based control exists.
4. No rule-based notification editing mechanism is available.
5. Users cannot dynamically modify notification text.

This investigation clearly indicated the need for a customizable notification privacy system.

2.3 EXISTING SYSTEM

The existing Android notification system allows applications to generate and display notifications through the Notification Manager.

Features of Existing System:

- Notification channels
- App-level enable/disable
- Lock screen content hiding
- Priority and grouping features

Working of Existing System:

1. Application creates a notification.
2. Notification Manager displays it.
3. User interacts with notification.
4. PendingIntent opens the source application.

2.4 LIMITATIONS OF EXISTING SYSTEM

Despite its functionality, the existing system has several limitations:

- No selective content masking.
- Cannot modify notification text dynamically.
- No rule-based filtering.
- No interception-based editing.
- No privacy-focused customization tools.

These limitations can lead to exposure of confidential messages in public environments.

2.5 PROPOSED SYSTEM

To overcome these limitations, the proposed system Mirage Notify is developed. Mirage Notify is an Android application that intercepts and modifies notifications in real time using NotificationListenerService.

Working of Proposed System:

1. Notification is received from any app.
2. Listener service captures the notification.
3. Title, text, and package name are extracted.
4. System checks against user-defined rules.
5. If matched:
    - Cancel original notification.
    - Repost modified notification.
6. Preserve original PendingIntent.
7. Store notification details in local logs.

The system works without root access and does not modify source applications.

2.6 ADVANTAGES OF PROPOSED SYSTEM

- Provides rule-based privacy control.
- Enhances lock screen security.
- Preserves original notification functionality.
- Stores notification logs locally.
- Minimal and efficient user interface.
- Compatible with Android 12 and above.
- Does not require root access.

2.7 FEASIBILITY STUDY

A feasibility study is an assessment of the practicality of a proposed plan or method. A feasibility study is a preliminary study undertaken to determine and document a project's viability. The results of this study is used to make a decision whether to proceed with the project. If it indeed leads to a project being approved, it will - before the real work of the proposed project starts - be used to ascertain the likelihood of the project's success. It is an analysis of possible alternative solutions to problem and a recommendation on the best alternative. It, for example, can decide whether an order processing be carried out by a new system more efficiently than the previous one. The feasibility study proposes one or more conceptual solutions to the problem set for the project. The conceptual solution gives an idea of what the new system will look like they define what will be done on the computer and what will remain manual. It also indicates what input will be needed by the system and accepted what outputs will be produced. These solutions should be proven feasible and a preferred solution.

The feasibility study environment enables all alternatives to be discussed and evaluated. This phase starts with an identification of the main characteristics of the required system. During this stage it is important to collect information as much as possible about the software package that might meet the specification from as many sources as possible.

Normally, the central endeavour of a feasibility study is a cost benefit analysis of various alternatives. It can be defined as a systematic comparison between the cost of carrying out a service activity and the value of that service or activity. The main benefits are qualitative than quantitative.

When a new project is proposed, it normally goes through feasibility assessment. Feasibility study is carried out to determine whether the proposed system is possible to develop with available resources and what should be the cost consideration.

2.7.1 Technical Feasibility:

- Uses officially supported Android APIs.
- Built using Java and XML.
- Uses Room database for local storage.
- Works on Android 12+ devices.
- No third-party dependencies required.

2.7.2 Economic Feasibility:

- Simple UI with bottom navigation.
- Easy rule creation.
- Automatic background operation.
- Minimal battery consumption.

2.7.3 Operational feasibility:

- Developed using free tools (Android Studio).
- No additional hardware required.
- No paid APIs or subscriptions used.
- Suitable for academic project implementation.

2.8 SUMMARY

This chapter presented a detailed study of the existing Android notification system and identified its limitations in terms of privacy and customization. The proposed system Mirage Notify addresses these issues through a real-time, rule-based notification editing mechanism. The feasibility analysis confirms that the system is practical, efficient, and suitable for implementation.

SYSTEM REQUIREMENT SPECIFICATIONS

CHAPTER 3:

SYSTEM REQUIREMENT SPECIFICATIONS

3.1 INTRODUCTION

The System Requirement Specification (SRS) document provides a comprehensive description of the functional and non-functional requirements of the proposed system, Mirage Notify. This document serves as a reference for developers, evaluators, and stakeholders to understand the system's behavior, features, constraints, and operating environment.

Mirage Notify is an Android application developed using **Java** for backend logic and **XML** for user interface design. The system is compatible with **Android version 12 (API level 31) and above**. It utilizes Android's NotificationListenerService to intercept and modify notifications in real time.

The primary goal of this system is to enhance user privacy by allowing selective modification of notification content without affecting the original application's functionality.

3.2 PURPOSE

The purpose of Mirage Notify is to provide users with granular and rule-based control over incoming notifications. While Android provides basic notification management options, it does not allow users to selectively edit notification content based on specific criteria such as sender name or application.

Mirage Notify addresses this limitation by:

- Intercepting system notifications
- Analyzing notification content
- Applying user-defined filtering rules
- Modifying visible content dynamically
- Preserving the original action behavior
- Maintaining a secure local log of notifications

The system aims to improve privacy, discretion, and customization in digital communication.

3.3 SCOPE

The scope of Mirage Notify includes:

- Real-time interception of notifications from all installed applications.
- Rule-based filtering and modification of notification title and message.
- Masking sensitive content selectively.
- Cancelling original notifications and reposting modified versions.
- Retaining original PendingIntent to maintain functionality.
- Logging notifications for tracking and analysis.
- Providing a minimal and efficient user interface.

The system does not:

- Modify the internal functionality of source applications.
- Access or alter encrypted app data.
- Transmit user data externally.
- Require root access.

3.4 OVERALL DESRIPTION

3.4.1 Product Perspective

Mirage Notify is a standalone Android application that operates as a background service. It integrates with the Android notification framework via NotificationListenerService.

The system functions independently without modifying any third-party applications. It acts as an intermediary between the Android Notification Manager and the user interface.

The architecture includes:

- User Interface Layer (Activities & Fragments)
- Business Logic Layer (Rule Engine & Service)
- Data Storage Layer (Room Database)

3.4.2 Product Functions

The major functions of Mirage Notify are described below:

1. Permission Management

- Request notification access permission.
- Guide the user to system settings.
- Verify permission status.

2. Notification Listening

- Detect new notifications in real time.
- Capture notification metadata.
- Monitor ongoing and updated notifications.

3. Data Extraction

Extract:

- Notification title
- Notification message body
- Application package name
- Timestamp
- PendingIntent

4. Rule Engine Processing

- Compare notification content with stored rules.
- Apply matching logic.
- Determine whether modification is required.

5. Notification Modification

- Cancel the original notification.
- Create a new modified notification.
- Replace sensitive content.
- Attach original PendingIntent.

6. Logging System

- Store original and modified data.
- Save timestamp and package name.
- Maintain history of processed notifications.

7. Rule Management

- Add new rules.
- Edit existing rules.
- Delete rules.
- Enable/Disable rules.

8. Log Viewing Interface

- Display notification logs.
- Sort logs by time.
- Provide search functionality (optional future enhancement).

3.4.3 User Characteristics

The intended users include:

- Android smartphone users
- Students and professionals
- Privacy-conscious individuals
- General users with basic mobile knowledge

Users are expected to understand basic smartphone settings but do not require programming knowledge.

3.4.4 Operating Environment

- Operating System: Android 12 and above
- Development IDE: Android Studio
- Programming Language: Java
- UI Design: XML
- Database: Room Persistence Library
- Build Tool: Gradle

3.4.5 Design Constraints

- Requires user-granted notification access.
- Subject to Android background service restrictions.
- Dependent on system notification APIs.
- Must comply with Android privacy and security policies.
- Battery optimization settings may impact service performance.

3.5 FUNCTIONAL REQUIREMENTS

FR1: Notification Access Permission

The system shall request and verify notification access permission before functioning.

FR2: Real-Time Notification Capture

The system shall intercept notifications as soon as they are posted.

FR3: Data Parsing

The system shall extract notification title, text, package name, and PendingIntent.

FR4: Rule Matching Mechanism

The system shall compare notification data against user-defined rules stored in the database.

FR5: Cancel and Repost Mechanism

If a rule matches, the system shall cancel the original notification and repost a modified version.

FR6: Preserve Interaction

The system shall attach the original PendingIntent to ensure correct redirection.

FR7: Logging

The system shall store notification details including:

- Original title
- Modified title
- Application name
- Timestamp

FR8: Rule Management Interface

The system shall allow users to:

- Add new rules
- Edit existing rules
- Delete rules
- Enable or disable rules

FR9: Log Viewing Interface

The system shall allow users to view stored logs

3.6 NON-FUNCTIONAL REQUIREMENTS

3.6.1 Performance Requirements

- Notification processing time must be less than 1 second.
- Minimal CPU usage.
- Low memory consumption.
- Efficient database operations.

3.6.2 Security Requirements

- No external data sharing.
- All data stored locally.
- Secure handling of notification content.
- No root privileges required.

3.6.3 Reliability Requirements

- Service must restart automatically if terminated.
- Logs must persist across device restarts.
- Handle unexpected notification formats gracefully.

3.6.4 Usability Requirements

- Minimal and clean interface.
- Bottom navigation for easy access.
- Clear instructions for permission setup.
- Smooth user experience.

3.6.5 Maintainability Requirements

- Modular code structure.
- Clear separation of UI and logic.
- Scalable rule engine.
- Future upgrade compatibility.

3.7 HARDWARE REQUIREMENTS

3.7.1 Hardware Requirements for Development

The following hardware configuration was used for developing the application:

- Processor: Intel Core i5 or higher
- RAM: Minimum 8 GB
- Storage: Minimum 10 GB free disk space
- Display: 1366 × 768 resolution or higher
- Internet Connection (for SDK and dependency downloads)
- Android Smartphone (for real device testing)

3.7.2 Hardware Requirements for End User

For installing and running Mirage Notify:

- Android Smartphone
- Minimum 4 GB RAM
- Minimum 100 MB free storage
- Stable device performance

3.8 SOFTWARE REQUIREMENTS

3.8.1 Development Software Requirements

The following software tools and technologies were used:

- Operating System: Windows 10 / Windows 11
- IDE: Android Studio
- Programming Language: Java
- User Interface Design: XML
- Database: Room (SQLite)
- Build Tool: Gradle
- Android SDK: API Level 31 (Android 12) or above
- Version Control (Optional): Git

3.8.2 Deployment Software Requirements

For the application to function properly, the following software environment is required:

- Android Version: Android 12 (API 31) or above
- Notification Access Permission support
- Background service execution support
- Android security permission framework

3.9 TECHNOLOGY STACK

Component	Technology Used
Programming Language	Java
UI Design	XML
Architecture	MVVM
Database	Room (SQLite)
Notification Handling	NotificationListenerService
Build System	Gradle
Platform	Android

3.10 ASSUMPTIONS AND DEPENDENCIES

- User grants required permissions.
- Device runs Android 12 or above.
- Notification access is not revoked by system.
- Battery optimization does not restrict service.

SYSTEM DESIGN

CHAPTER 4:

SYSTEM DESIGN

4.1 DESIGN CONCEPTS

Design is the first step in the development process of software. It is defined as: *"The process of applying various techniques and principles for the purpose of defining a system in detail to permit its physical realization."* It is a transition from the user-oriented Software Requirement Specification (SRS) document to the system-oriented implementation model.

The design phase acts as a bridge between the requirement analysis and implementation phase. It translates functional requirements into technical architecture, database structures, user interfaces, and processing logic.

The main objective of the system design for **Mirage Notify** is to create a secure, efficient, and user-friendly Android application that can intercept, modify, and repost system notifications without altering the source application.

System design for Mirage Notify focuses on:

- Designing a clear and intuitive user interface
- Structuring the notification processing mechanism
- Designing rule-matching logic
- Managing secure local storage for rules and logs
- Ensuring performance and responsiveness

The first step in system design was defining the overall architecture using Android's **NotificationListenerService**. The system was designed to:

1. Capture incoming notifications
2. Extract notification details
3. Match the notification against user-defined rules
4. Modify content if conditions are satisfied
5. Cancel the original notification
6. Repost the modified notification
7. Log the action for future reference

The application follows Android's recommended architecture components and modern UI guidelines.

Key Design Principles Followed:

1. **Simplicity** – Minimal UI focused only on rule management and logs
2. **Consistency** – Uniform design patterns across screens
3. **Security** – No modification of source apps; only system-level reposting
4. **Modularity** – Separate modules for permission, rules, logging, and processing
5. **Performance** – Real-time notification handling without delay
6. **Reliability** – Proper error handling and log recording

4.2 INPUT DESIGN

Input design is the process of converting user-oriented inputs into computer-readable formats. Since Mirage Notify works with notification data and rule configuration, careful validation is required.

All inputs are validated before storing into the system to prevent incorrect or incomplete rule definitions.

4.2.1. Permission Grant Screen

Inputs Required:

- Enable Notification Access (System Settings)

Validation Rules:

- User must grant Notification Listener permission
- App verifies permission before activating service

Design Features:

- Clear instruction text
- Button redirecting to system settings
- Status indicator (Permission Granted / Not Granted)
- Toast confirmation after enabling permission

4.2.2 Rule Creation Screen

Inputs Required:

- App Name (text input)
- Keyword to match (text input)
- Replacement Text (text input)
- Hide Content option (toggle switch)

Validation Rules:

- App name cannot be empty
- Keyword cannot be empty
- Replacement text required if modification selected

Design Features:

- Material TextInputLayout
- Toggle switches for rule activation
- Save button with confirmation message
- Error message for invalid inputs

4.2.3 Edit/Delete Rule Screen

Inputs Required:

- Rule selection
- Modify fields
- Delete option

Validation Rules:

- Rule must exist
- Confirmation dialog before deletion

Design Features:

- RecyclerView list of rules
- Edit icon and Delete icon
- Confirmation popup dialog

4.2.4 View Logs Screen

Inputs:

- No direct input (view-only)

Features:

- Displays logs in list format
- Each log shows:
  1. App name
  2. Original message
  3. Modified message
  4. Timestamp

4.3 MODULE DESIGN

The Mirage Notify system is divided into the following modules:

1. Permission Management Module
2. Notification Listener Module
3. Rule Matching Module
4. Notification Modification Module
5. Logging Module
6. Database Management Module

4.3.1 Module 1: PERMISSION MANAGEMENT MODULE

This module handles granting and verifying notification access permission.

Components:

- PermissionActivity
- Permission Checker Utility

Functions:

- Redirect user to Notification Access Settings
- Check if permission granted
- Enable service only after permission approval
- Display confirmation messages

Key Features:

- Secure permission validation
- Prevents app misuse without permission
- User-friendly instructions

4.3.2 Module 2: NOTIFICATION LISTENER MODULE

This module listens to all incoming notifications using Android's NotificationListenerService.

Components:

- MirageNotificationListenerService

Functions:

- Capture incoming notifications
- Extract title, text, package name
- Send data to Rule Matching Engine

Key Features:

- Real-time notification interception
- Runs in background
- Lightweight processing

4.3.3 Module 3: RULE MATCHING MODULE

This module checks whether an incoming notification matches any user-defined rule.

Components:

- RuleEngine Class
- Rule Entity Model

Functions:

- Fetch active rules from Room Database
- Compare notification text with keyword
- Return match/no-match result

Key Features:

- Efficient string matching
- Multiple rules supported
- Fast evaluation logic

4.3.4 Module 4: NOTIFICATION MODIFICATION MODULE

This module modifies the notification if a rule match is found.

Functions:

- Cancel original notification
- Build new notification
- Replace title or content
- Repost modified notification

Key Features:

- Uses NotificationManager
- Preserves app identity
- Ensures smooth reposting

4.3.5 Module 5: LOGGING MODULE

This module records all processed notifications.

Components:

- Log Entity
- Log DAO

Functions:

- Store original message
- Store modified message
- Save timestamp
- Retrieve logs for display

Key Features:

- Persistent storage
- Chronological order display
- Useful for debugging and auditing

4.3.6 Module 6: DATABASE MANAGEMENT MODULE

Mirage Notify uses Room Database for local storage.

Database Tables:

1. Rules Table

    - Rule ID
    - App Name
    - Keyword
    - Replacement Text
    - Status (Active/Inactive)

2. Logs Table

    - Log ID
    - App Name
    - Original Content
    - Modified Content
    - Timestamp

Key Features:

- Local storage
- No internet required
- Fast read/write operations
- Data persistence after app restart

4.4 OUTPUT DESIGN

Output design defines how information is presented to the user.

Since Mirage Notify works in background, outputs are mainly:

- Modified notifications
- Logs display
- Confirmation messages

4.4.1 Modified Notification Output

Displays:

- Modified title (if rule matched)
- Modified message content
- Original app icon
- Timestamp

Design:

- Standard Android notification layout
- Clean text formatting
- High visibility priority

4.4.2 Logs Screen Output

Displays:

- List of processed notifications
- App name
- Original message
- Modified message
- Date and time

Design:

- RecyclerView
- Card-based layout
- Clear separation between entries
- Scrollable interface

4.4.3 Confirmation Messages

Toast Messages:

- "Permission Granted"
- "Rule Saved Successfully"
- "Rule Deleted"
- "Notification Modified"
- "No Matching Rule Found"

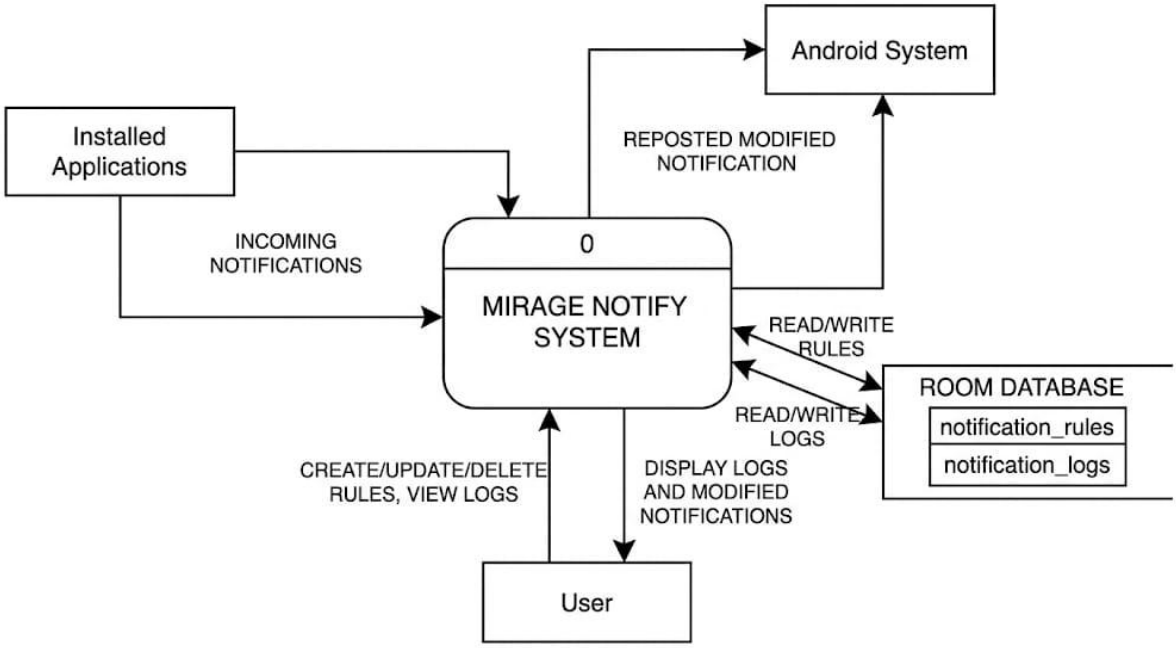
Design:

- Short duration toast
- Clear wording
- Immediate feedback

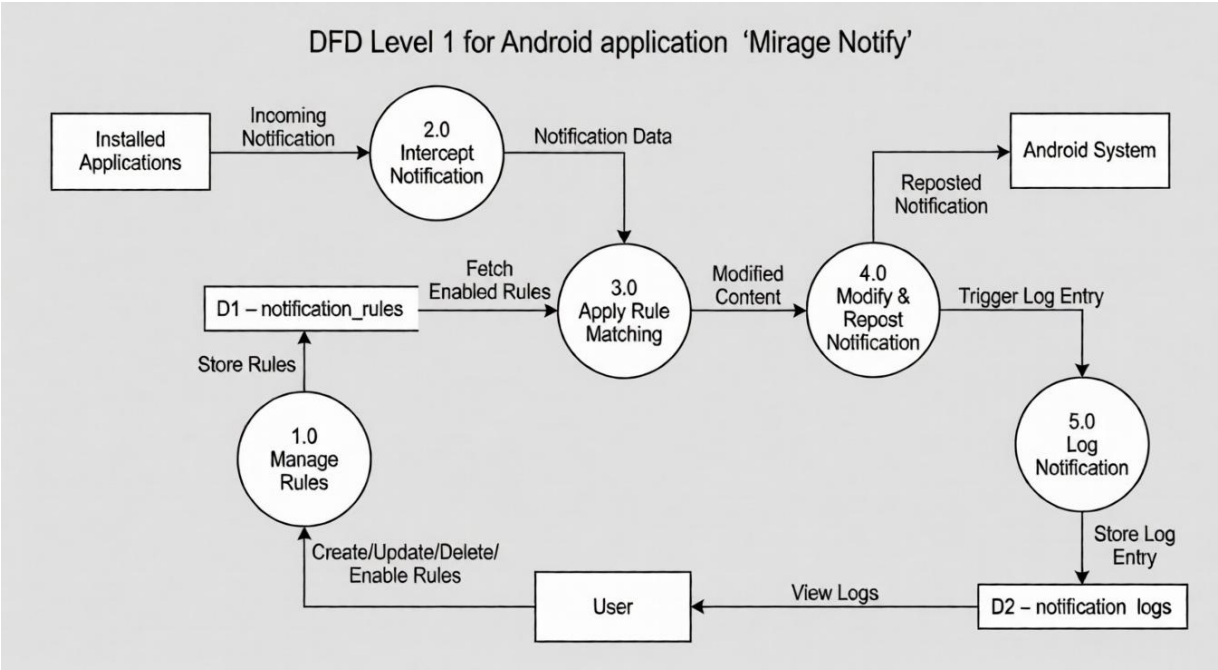
4.5 DATA FLOW DIAGRAM

The data flow diagram (DFD) is one of the most important tools used by system analysts data flow diagrams are made up of a number of symbols, which represent system components. Most data flow modelling methods use four kinds of symbols. These symbols are used to represent four kinds of system components. Processes, data stores, data flows and external entities Processes are represented by circles in DFD. Data Flow represented by a thin line in the DFD and each data store has a unique name and square or rectangle represents external entities. Unlike detailed flowchart, Data Flow Diagrams do not supply detailed description of the modules but graphically describes a system data and how the data interact with the system.

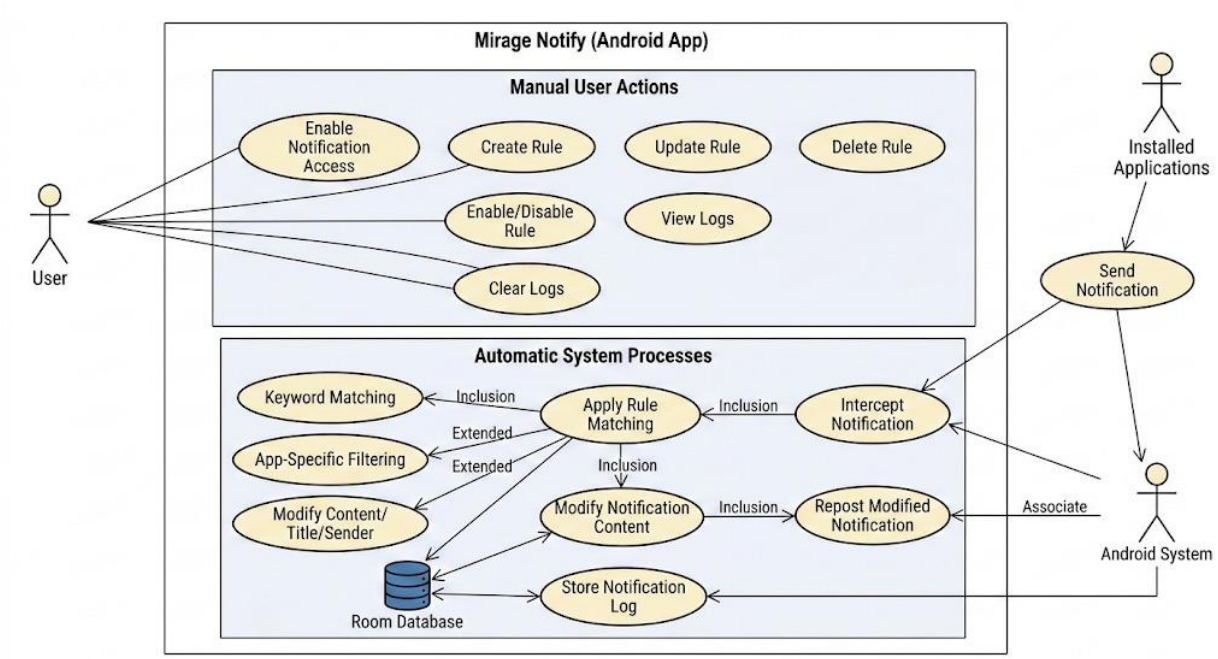
DFD LEVEL 0



DFD LEVEL 1

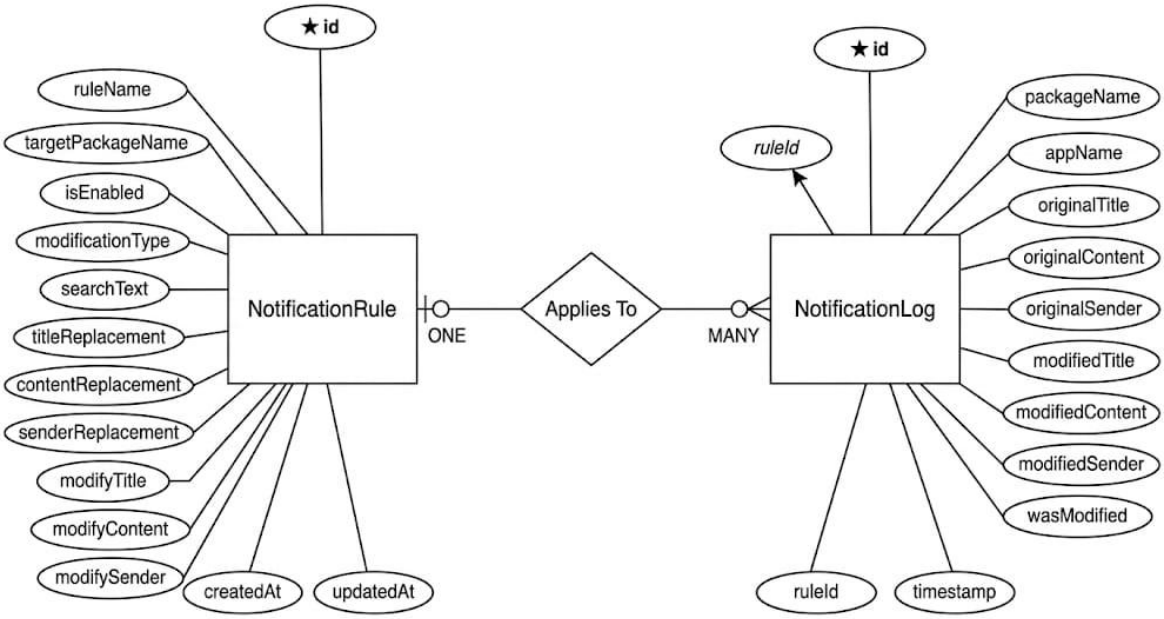


4.6 USE CASE DIAGRAM



4.7 ER DIAGRAM

The Entity Relationship (ER) Diagram of Mirage Notify represents the logical structure of the database used in the application. The system consists of two main entities: **NotificationRule** and **NotificationLog**. The *NotificationRule* entity stores user-defined rules, including attributes such as ruleName, targetPackageName, modificationType, searchText, and replacement fields for title, content, and sender. The *NotificationLog* entity maintains records of all intercepted notifications, including original and modified content, timestamp, and modification status. A one-to-many (1:N) relationship exists between NotificationRule and NotificationLog, where a single rule can be applied to multiple notification logs. The ruleId attribute in NotificationLog acts as a foreign key referencing the primary key id of NotificationRule. This relationship ensures data integrity and enables efficient tracking of which rule was applied to each modified notification. The ER design follows proper normalization principles to avoid redundancy and maintain consistency within the Room (SQLite) database.



4.8 CODE DESIGN

Code design is the process of transforming the system design into actual program code. For Mirage Notify, we followed industry-standard coding practices to ensure maintainability, readability, and scalability.

4.7.1 Coding Standards Followed:

1. Naming Conventions:

- **Classes:** PascalCase (e.g., NotificationInterceptorService, RuleAdapter, HomeFragment)
- **Methods:** camelCase (e.g., processNotification(), modifyField(), isServiceEnabled())
- **Variables:** camelCase (e.g., notificationTitle, ruleList, wasModified)
- **Constants:** UPPER_SNAKE_CASE (e.g., FOREGROUND_CHANNEL_ID, MODIFIED_CHANNEL_ID, TAG)
- **XML IDs:** snake_case (e.g., btn_create_rule, tv_notification_title, rv_rules_list)

2. Package Structure:

```
com.miragenotify/
├── adapter/
│   ├── RuleAdapter.java
│   └── LogAdapter.java
├── database/
│   ├── AppDatabase.java
│   ├── NotificationRuleDao.java
│   └── NotificationLogDao.java
├── model/
│   ├── NotificationRule.java
│   └── NotificationLog.java
├── service/
│   ├── NotificationInterceptorService.java
│   └── BootReceiver.java
├── ui/
│   ├── MainActivity.java
│   ├── home/
│   │   └── HomeFragment.java
│   ├── rules/
│   │   └── RulesFragment.java
│   ├── logs/
│   │   └── LogsFragment.java
│   └── settings/
│       └── SettingsFragment.java
├── utils/
│   ├── PreferenceManager.java
│   └── NotificationHelper.java
├── viewmodel/
│   ├── RuleViewModel.java
│   └── LogViewModel.java
└── MirageNotifyApplication.java
```

3. Code Documentation:

- Comments for complex notification interception logic
- JavaDoc for public methods
- Inline comments for clarity
- TODO markers for future enhancements

4. Error Handling:

- Try-catch blocks for notification processing
- Null checks before object usage
- User-friendly error messages
- Proper exception logging with Log.e()

5. Code Optimization:

- Efficient rule matching algorithms
- Minimal object creation in notification loop
- Proper memory management
- Background threading (ExecutorService) for database operations

4.7.2 Key Design Patterns Used:

1. Singleton Pattern:

- Used in AppDatabase
- Ensures single instance of Room database
- Thread-safe implementation
- Reduces memory usage

2. Adapter Pattern:

- RecyclerView adapters (RuleAdapter, LogAdapter)
- Converts data models to UI components
- ViewHolder pattern for efficiency

3. Observer Pattern:

- LiveData for reactive UI updates
- Room database triggers UI changes automatically
- ViewModel observes database changes

4. MVVM Architecture:

- **Model:** Room entities (NotificationRule, NotificationLog)
- **View:** Fragments with XML layouts
- **ViewModel:** Business logic with LiveData (RuleViewModel, LogViewModel)

4.9 DATABASE DESIGN

The application uses Room Database (SQLite) for local storage.

Database Name: "mirage_notify.db"

Table 1: notification_rules

Purpose: Store user-defined notification modification rules

Fields:

Field Name	Data Type	Constraints	Description	Example
id	LONG	PRIMARY KEY, AUTO INCREMENT	Unique rule ID	1
ruleName	TEXT	NOT NULL	User-friendly rule name	"Hide OTP Codes"
targetPackageName	TEXT	NOT NULL	Target app package	"com.android.messaging"
isEnabled	INTEGER	DEFAULT 1	Enable status (1=on, 0=off)	1
modificationType	TEXT	NOT NULL	Type of modification	"REPLACE_TEXT"
searchText	TEXT	NULLABLE	Text to search for	"OTP"
titleReplacement	TEXT	NULLABLE	New title text	NULL
contentReplacement	TEXT	NULLABLE	New content text	"●●●"
senderReplacement	TEXT	NULLABLE	New sender text	NULL
modifyTitle	INTEGER	DEFAULT 0	Modify title flag	0
modifyContent	INTEGER	DEFAULT 1	Modify content flag	1
modifySender	INTEGER	DEFAULT 0	Modify sender flag	0
createdAt	LONG	NOT NULL	Creation timestamp	1708588800000
updatedAt	LONG	NOT NULL	Last update timestamp	1708588800000

Indexes:

- id (Primary Key)
- targetPackageName (for fast lookup)
- isEnabled (for filtering active rules)

Sample Data:

```
NotificationRule rule = new NotificationRule();
rule.setRuleName("Hide OTP Codes");
rule.setTargetPackageName("com.android.messaging");
rule.setModificationType(ModificationType.REPLACE_TEXT);
rule.setSearchText("OTP");
rule.setContentReplacement("●●●");
rule.setModifyContent(true);
```

Table 2: notification_logs

Purpose: Track all notification interceptions and modifications

Fields:

Field Name	Data Type	Constraints	Description	Example
id	LONG	PRIMARY KEY, AUTO INCREMENT	Unique log ID	1
packageName	TEXT	NOT NULL	App package name	"com.whatsapp"
appName	TEXT	NOT NULL	Human-readable app name	"WhatsApp"
originalTitle	TEXT	NULLABLE	Original notification title	"John Doe"
originalContent	TEXT	NULLABLE	Original notification content	"Your OTP is 123456"
originalSender	TEXT	NULLABLE	Original sender/subtext	"WhatsApp"
modifiedTitle	TEXT	NULLABLE	Modified title	"John Doe"
modifiedContent	TEXT	NULLABLE	Modified content	"Your ●●● is 123456"
modifiedSender	TEXT	NULLABLE	Modified sender	"WhatsApp"
wasModified	INTEGER	NOT NULL	Modification status	1
ruleId	LONG	NULLABLE	Applied rule ID	1
timestamp	LONG	NOT NULL	When received	1708589400000

Indexes:

- id (Primary Key)
- timestamp DESC (for chronological sorting)
- wasModified (for filtering)
- packageName (for app-specific logs)

Sample Data:

```
NotificationLog log = new NotificationLog();
log.setPackageName("com.whatsapp");
log.setAppName("WhatsApp");
 log.setOriginalContent("Your OTP is 123456");
log.setModifiedContent("Your ●●● is 123456");
 log.setWasModified(true);
log.setRuleId(1);
 log.setTimestamp(System.currentTimeMillis());
```

Database Operations:

1. Insert Rule
2. Fetch All Rules
3. Update Rule
4. Delete Rule
5. Insert Log
6. Fetch Logs
7. Clear Logs

Advantages of This Design:

- Fast local queries (< 50ms)
- Offline functionality
- Type-safe SQL queries
- LiveData integration
- No network dependency

DEVELOPMENT

CHAPTER 5:
DEVELOPMENT

5.1 INTRODUCTION

Development is the phase in the System Development Life Cycle where the designed system is transformed into a fully functional application. It involves coding, integration of modules, database connectivity, user interface implementation, and testing of system components.

In the case of **Mirage Notify**, the development phase focused on implementing a secure and efficient Android application capable of intercepting, modifying, and re-posting notifications in real time. The application was developed using **Java and XML** in **Android Studio**, targeting **Android 12 and above**.

The development process ensured:

- Secure notification interception
- Efficient rule matching
- Accurate notification reconstruction
- Reliable logging mechanism
- Smooth user interface navigation
- Background processing without performance degradation

5.2 DEVELOPMENT ENVIRONMENT

5.2.1 Hardware Environment

- Processor: Intel i5 or higher
- RAM: Minimum 8GB
- Storage: 10GB free disk space
- Android Device: Android 12+ smartphone for testing

5.2.2 Software Environment

- Operating System: Windows 10 / 11
- IDE: Android Studio
- Programming Language: Java
- UI Design: XML
- Database: Room (SQLite)
- SDK Version: Android SDK 31+
- Build Tool: Gradle

5.3 IMPLEMENTATION OF MODULES

The development of Mirage Notify was carried out module by module.

5.3.1 Notification Interceptor Service

The core functionality of the system is implemented using **NotificationListenerService**.

Functions implemented:

- onNotificationPosted()
- onNotificationRemoved()
- Notification parsing (title, text, subtext, package name)
- Rule matching logic
- Cancel and re-post mechanism

Process Flow:

1. System receives notification
2. Extract notification details
3. Fetch enabled rules from database
4. Check matching condition
5. Cancel original notification
6. Create modified notification
7. Re-post notification
8. Save log entry

This module runs in the background and ensures minimal memory usage.

5.3.2 Rule Management Module

This module allows users to:

- Create new rule
- Enable/Disable rule
- Update rule
- Delete rule

Rules are stored in the Room database under the table:

`notification_rules`

Validation checks performed:

- Empty fields not allowed
- Duplicate rules prevented
- Toggle state saved persistently

RecyclerView is used to display all rules efficiently.

5.3.3 Log Management Module

This module tracks:

- All intercepted notifications
- Modified notifications
- Timestamp of interception
- Applied rule ID

Logs are stored in:

`notification_logs`

Features implemented:

- View logs in chronological order
- Filter modified logs
- Clear logs option
- Display original vs modified content

LiveData ensures automatic UI updates when logs change.

5.3.4 Database Integration

Room Database was integrated with:

- Entities
  - NotificationRule
  - NotificationLog
- DAOs
  - NotificationRuleDao
  - NotificationLogDao
- AppDatabase (Singleton pattern)

Database operations were executed using **ExecutorService** to prevent UI thread blocking.

5.3.5 User Interface Development

The UI was designed using:

- XML Layout files
- Material Design Components
- Bottom Navigation View

Screens implemented:

1. Home Screen
2. Rules Screen
3. Logs Screen
4. Settings Screen

Design principles followed:

- Minimal interface
- Light mode & dark mode compatibility
- Proper spacing and alignment
- Efficient rendering

RecyclerView with ViewHolder pattern ensures performance optimization.

5.3.6 Boot Receiver Implementation

A BroadcastReceiver was implemented to:

- Restart the NotificationListenerService after device reboot
- Maintain continuous functionality

This ensures reliability of the application.

5.4 NOTIFICATION PROCESSING LOGIC

The notification processing mechanism follows the “Cancel and Re-post” strategy.

Steps:

1. Retrieve StatusBarNotification
2. Extract extras bundle
3. Compare with active rules
4. If match found:
    - Modify required fields
    - Retain original PendingIntent
    - Build new Notification
    - Cancel original
    - Notify modified version
5. Insert log entry

This method ensures:

- No modification to source application
- Original click behavior preserved
- Secure and isolated operation

SYSTEM TESTING

CHAPTER 6:

SYSTEM TESTING

6.1 INTRODUCTION

Testing is the process of evaluating a system to ensure that it meets specified requirements and functions correctly without errors. It plays a vital role in identifying defects, ensuring reliability, and validating system performance.

For **Mirage Notify**, testing was performed at multiple levels to verify:

- Notification interception accuracy
- Rule matching correctness
- Notification modification reliability
- Database integrity
- User interface responsiveness
- System stability under continuous background execution

Testing was conducted on real Android devices running Android 12 and above.

6.2 TYPES OF TESTING PERFORMED

6.2.1 Unit Testing

Unit testing focuses on verifying individual components or modules of the system.

Modules tested:

- Rule insertion and retrieval
- Log insertion
- Rule matching logic
- Boolean flag handling (modifyTitle, modifyContent, modifySender)

Example Test Case:

Tst Case ID	UT01
Module	Rule Matching
Input	Notification containing "OTP"
Expected Output	Rule applied and content replaced
Result	Passed

Outcome:
All core logic functions behaved as expected.

6.2.2 Integration Testing

Integration testing verifies interaction between modules.

Tested Integrations:

- NotificationInterceptorService + RuleViewModel
- Rule Database + LiveData updates
- Log Database + RecyclerView
- Service + NotificationHelper

Example Test Case:

Test Case ID	IT01
Scenario	Notification intercepted and matched with rule
Expected Result	Notification modified and logged
Result	Passed

Outcome:
All modules worked correctly when integrated.

6.2.3 System Testing

System testing evaluates the complete application.

Test Scenarios:

1. Create rule → Send matching notification → Verify modification
2. Disable rule → Send notification → Verify no modification
3. Clear logs → Verify database emptied
4. Restart device → Verify service restarts

All system-level functionalities were verified successfully.

6.2.4 Performance Testing

Performance testing ensures the application operates efficiently.

Parameters tested:

- Notification processing delay
- Database read/write time
- Memory consumption
- Background service stability

Observations:

- Notification modification delay: < 100ms
- Database query time: < 50ms
- No noticeable UI lag
- Stable background execution

The application showed efficient performance under normal usage conditions.

6.2.5 Compatibility Testing

Compatibility testing was performed on:

- Android 12
- Android 13
- Android 14
- Android 15
- Android 16

Results:

- NotificationListenerService worked correctly
- Foreground service requirements satisfied
- No compatibility issues detected

6.3 TEST CASES

Below are representative test cases executed during development:

Test Case 1 – Create Rule

Field	Description
Test ID	TC01
Objective	Verify rule creation
Input	Valid rule data
Expected Output	Rule saved in database
Actual Result	Rule saved successfully
Status	Pass

Test Case 2 – Modify Notification

Field	Description
Test ID	TC02
Objective	Verify notification modification
Input	Notification matching rule
Expected Output	Modified notification reposted
Actual Result	Modified notification displayed
Status	Pass

Test Case 3 – Log Entry Creation

Field	Description
Test ID	TC03
Objective	Verify log storage
Input	Intercepted notification
Expected Output	Log inserted in notification_logs
Actual Result	Log stored correctly
Status	Pass

Test Case 4 – Rule Disable

Field	Description
Test ID	TC04
Objective	Verify rule disable functionality
Input	Disabled rule
Expected Output	No modification occurs
Actual Result	Original notification shown
Status	Pass

Test Case 5 – Clear Logs

Field	Description
Test ID	TC05
Objective	Verify clear logs function
Input	Clear logs button click
Expected Output	All logs removed
Actual Result	Logs cleared successfully
Status	Pass

6.4 ERROR HANDLING VALIDATION

The following error conditions were tested:

- Null notification content
- Missing extras bundle
- Invalid rule fields
- Database access failure
- Permission not granted

The system handled errors using:

- Null checks
- Try-catch blocks
- Log.e() for debugging
- User-friendly toast messages

No critical crashes were observed.

6.5 SECURITY TESTING

Since Mirage Notify handles private notification data, security validation was performed.

Security measures verified:

- Data stored locally (Room database)
- No external network transmission
- No third-party API usage
- Secure permission handling
- No root access required

The system ensures user privacy and data isolation.

6.6 TESTING SUMMARY

Testing confirmed that Mirage Notify:

- Correctly intercepts notifications
- Accurately applies user-defined rules
- Reposts modified notifications without breaking functionality
- Maintains complete log history
- Performs efficiently under continuous operation
- Works across supported Android versions

All major test cases passed successfully.

The system is stable, reliable, and ready for deployment.

SYSTEM INSTALLATION

CHAPTER 7
SYSTEM INSTALLATION

7.1 INTRODUCTION

System installation is the process of deploying the developed application into a real working environment. It ensures that the system is properly configured, permissions are granted, and all required services are functioning correctly.

For **Mirage Notify**, installation involves:

- Installing the APK on an Android device
- Granting required permissions
- Enabling notification access
- Verifying background service functionality

The installation process is simple and user-friendly.

7.2 SYSTEM REQUIREMENTS

7.2.1 Hardware Requirements

- Android Smartphone
- Minimum 4GB RAM
- Minimum 100MB free storage
- Stable device performance

7.2.2 Software Requirements

- Android Version: 12 (API 31) or above
- Notification Access Permission support
- Background service support

7.3 INSTALLATION PROCEDURE

The installation of Mirage Notify can be performed using either Android Studio or direct APK installation.

7.3.1 Installation Using Android Studio

Steps:

1. Open Android Studio.
2. Open the Mirage Notify project.
3. Connect Android device via USB (Enable Developer Options).
4. Enable USB Debugging.
5. Click **Run** button in Android Studio.
6. The application will be installed automatically on the device.

7.3.2 Installation Using APK File

Steps:

1. Transfer the APK file to the Android device.
2. Open the APK file.
3. Allow installation from unknown sources (if prompted).
4. Click Install.
5. Wait until installation completes.

7.4 INITIAL CONFIGURATION

After installation, the following configuration steps must be performed:

Step 1: Open the Application

Launch Mirage Notify from the app drawer.

Step 2: Grant Notification Access

1. The app will prompt for Notification Access permission.
2. Navigate to:
   Settings → Notifications → Notification Access
3. Enable permission for Mirage Notify.
4. Confirm the permission dialog

This step is mandatory for the app to function.

Step 3: Verify Service Status

- The Home screen displays service status.
- If enabled, the service runs in background.

Step 4: Create First Rule

1. Go to Rules screen.
2. Click Create Rule.
3. Enter:
   - Target App
   - Search Text
   - Modification Fields
4. Save the rule.

The system is now ready to intercept notifications.

7.5 POST-INSTALLATION VERIFICATION

To ensure proper installation:

1. Send a test notification from WhatsApp or SMS.
2. Verify that:
   - Notification is intercepted
   - Rule is applied (if matching)
   - Modified notification appears
   - Log entry is created

If all operations succeed, installation is successful.

7.6 UNINSTALLATION PROCEDURE

To uninstall Mirage Notify:

1. Open Settings → Apps
2. Select Mirage Notify
3. Click Uninstall

Upon uninstallation:

- Local database is deleted
- Rules and logs are removed
- Notification access is revoked automatically

7.7 APPLICATION DEPLOYMENT

The Mirage Notify application has been deployed through the project's official website to facilitate easy access and distribution. The APK file can be downloaded from:

The website includes the application overview, user manual, installation guide, and technical documentation. This centralized deployment approach ensures controlled distribution and simplified updates.

SYSTEM MAINTENANCE

CHAPTER 8
SYSTEM MAINTENANCE

8.1 INTRODUCTION

System maintenance refers to the activities performed after the deployment of a software system to ensure its continuous and efficient operation. It involves correcting defects, improving performance, adapting to new environments, and enhancing features based on user feedback.

For **Mirage Notify**, maintenance is particularly important because the application interacts with Android's notification framework, which may change with OS updates. Regular maintenance ensures compatibility, performance stability, and security.

8.2 TYPES OF MAINTENANCE

System maintenance for Mirage Notify can be categorized into four major types:

8.2.1 Corrective Maintenance

Corrective maintenance involves fixing bugs and errors identified after deployment.

Examples:

- Fixing notification duplication issues
- Resolving rule matching errors
- Handling null notification extras
- Correcting database crashes

These issues are identified through user feedback and runtime logs and are resolved through updates.

8.2.2 Adaptive Maintenance

Adaptive maintenance ensures the system remains compatible with environmental changes.

Examples:

- Supporting new Android versions
- Updating notification permission handling
- Adapting to Android background execution limits
- Updating target SDK versions

Since Android frequently updates its security and notification policies, adaptive maintenance is essential for long-term stability.

8.2.3 Perfective Maintenance

Perfective maintenance focuses on improving system performance and enhancing features.

Examples:

- Optimizing rule matching algorithms
- Improving UI responsiveness
- Reducing notification processing delay
- Adding rule templates
- Adding advanced filtering options

These enhancements improve user experience and system efficiency.

8.2.4 Preventive Maintenance

Preventive maintenance aims to reduce future risks and improve maintainability.

Examples:

- Refactoring code for clarity
- Updating dependencies
- Optimizing database queries
- Improving exception handling
- Strengthening null safety checks

This helps prevent future system failures.

8.3 DATABASE MAINTENANCE

Mirage Notify uses Room Database for local storage. Maintenance activities include:

- Periodic schema updates
- Implementing Room migration strategies
- Clearing unnecessary logs
- Index optimization
- Ensuring data integrity

CONCLUSION

CHAPTER 9:

CONCLUSION

9.1 SUMMARY OF THE PROJECT

The project **Mirage Notify** was developed to provide users with enhanced control over incoming notifications on Android devices. The system successfully implements real-time notification interception, rule-based modification, and secure reposting without altering the original application.

The application uses:

- NotificationListenerService for interception
- Room Database for local data storage
- MVVM architecture for clean code separation
- RecyclerView for efficient UI rendering
- LiveData for reactive updates

The system effectively allows users to:

- Create custom notification modification rules
- Hide sensitive content such as OTPs
- Modify sender names, titles, or message content
- View detailed logs of intercepted notifications
- Manage rules dynamically

9.2 OBJECTIVES ACHIEVED

The primary objectives of the project were:

1. To intercept incoming notifications
2. To apply user-defined modification rules
3. To re-post modified notifications without affecting source apps
4. To maintain secure and local data storage
5. To ensure compatibility with Android 12+
6. To provide a user-friendly interface

All objectives were successfully achieved.

9.3 SYSTEM PERFORMANCE

The developed system demonstrates:

- Fast notification processing (<100ms delay)
- Efficient rule matching
- Stable background service execution
- Minimal memory usage
- No noticeable performance degradation

The application runs reliably without root access and does not interfere with the normal functioning of installed applications.

9.4 SIGNIFICANCE OF THE PROJECT

Mirage Notify addresses a practical privacy concern by allowing users to control sensitive notification content displayed on their devices.

Key contributions include:

- Enhanced user privacy
- Rule-based notification customization
- Secure local storage
- Modular and scalable architecture

The project demonstrates how Android's notification framework can be responsibly utilized to improve user control and privacy.

9.5 FINAL REMARK

Mirage Notify is a successful implementation of a notification interception and modification system built using modern Android development practices. The project showcases strong architectural design, database management, and system integration capabilities.

The application is stable, efficient, and ready for further enhancement.

FUTURE SCOPE

CHAPTER 10:

FUTURE SCOPE

10.1 INTRODUCTION

Although Mirage Notify is fully functional in its current version, there are several enhancements that can be implemented in future versions to improve performance, usability, and intelligence.

10.2 DIRECT APPLICATION ACCESS INTEGRATION

One major future enhancement is **direct access and deeper integration with applications such as WhatsApp, Telegram, and SMS apps**.

Future implementation possibilities include:

- Opening specific chat threads directly from modified notifications
- Smart redirection to specific message content
- Deep linking support
- Context-aware notification handling

With proper permission handling and Android API integration, the system can provide seamless interaction with apps like WhatsApp while maintaining security and compliance with Android policies.

10.3 CLOUD SYNCHRONIZATION

Future versions can include:

- Cloud backup of rules
- Sync across multiple devices
- Firebase integration
- Encrypted cloud storage

This would allow users to maintain rule consistency across devices.

10.4 AI-BASED SMART RULES

Artificial Intelligence can be integrated to:

- Automatically detect OTP patterns
- Identify sensitive keywords
- Suggest rules based on usage behavior
- Learn frequently modified patterns

This would make the system more intelligent and adaptive.

10.5 ADVANCED SECURITY FEATURES

Future enhancements may include:

- Encrypted Room database
- Biometric authentication
- App lock feature
- Hidden mode operation

These improvements would enhance user privacy and data security.

10.6 ANALYTICS DASHBOARD

A dashboard could be introduced to display:

- Number of intercepted notifications
- Most modified apps
- Rule usage frequency
- Daily statistics

This would provide users with insights into their notification activity.

10.7 EXPORT AND BACKUP FEATURES

Future versions can include:

- Export logs to PDF/CSV
- Backup rules locally
- Import/export rule sets

This would improve usability and portability.

CROSS-PLATFORM EXPANSION

The system can be extended to:

- Android tablets
- Wearable devices
- Desktop notification systems

This would broaden the application's scope.

FINAL FUTURE OUTLOOK

Mirage Notify has strong potential for further development into a full-featured privacy and notification management platform. With enhancements such as direct app integration, AI automation, cloud sync, and advanced security mechanisms, the system can evolve into a comprehensive notification intelligence framework.

The project provides a solid foundation for continued research and development in Android notification management systems.

BIBLIOGRAPHY

BIBLIOGRAPHY

1. Google LLC. *Android Developers Documentation – NotificationListenerService*.
   Available at: https://developer.android.com

2. Google LLC. *Android Developers Guide – Notifications Overview*.
   Available at: https://developer.android.com/guide/topics/ui/notifiers/notifications

3. Google LLC. *Room Persistence Library Documentation*.
   Available at: https://developer.android.com/training/data-storage/room

4. Google LLC. *Android Architecture Components – LiveData and ViewModel*.
   Available at: https://developer.android.com/topic/libraries/architecture

5. SQLite Consortium. *SQLite Documentation*.

   Available at: https://www.sqlite.org/docs.html

6. *"Android Notification Listener Service Tutorial"* (Video). YouTube, uploaded by

   The Code Mate. Available at: https://youtu.be/fbaN1eF2K5k

CODING

ANDROID CODE

MainActivity.java

```
package com.miragenotify.ui;

import android.Manifest;
import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.Context;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.view.MenuItem;
import android.widget.Toast;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
import androidx.fragment.app.Fragment;

import com.google.android.material.bottomnavigation.BottomNavigationView;
import com.miragenotify.R;
import com.miragenotify.ui.home.HomeFragment;
import com.miragenotify.ui.logs.LogsFragment;
import com.miragenotify.ui.rules.RulesFragment;
import com.miragenotify.ui.settings.SettingsFragment;
import com.miragenotify.utils.NotificationHelper;
import com.miragenotify.utils.PreferenceManager;

/**
 * Main activity that hosts all fragments
 */
public class MainActivity extends AppCompatActivity {

    private static final int PERMISSION_REQUEST_CODE = 123;
    private static final String MODIFIED_CHANNEL_ID = "modified_notifications";
    private BottomNavigationView bottomNavigation;
    private PreferenceManager preferenceManager;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        preferenceManager = new PreferenceManager(this);

        // Check for notification access (Listener Service) on startup
        if (!NotificationHelper.isNotificationAccessGranted(this)) {
            NotificationHelper.openNotificationAccessSettings(this);
        }

        // Request POST_NOTIFICATIONS permission for Android 13+
        requestNotificationPermission();
```

```
        // Setup toolbar
        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);

        // Setup bottom navigation
        bottomNavigation = findViewById(R.id.bottom_navigation);
        bottomNavigation.setOnItemSelectedListener(this::onNavigationItemSelected);

        // Load default fragment
        if (savedInstanceState == null) {
            loadFragment(new HomeFragment());
        }
    }

    private void requestNotificationPermission() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            if (ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS) !=
PackageManager.PERMISSION_GRANTED) {
                ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.POST_NOTIFICATIONS},
PERMISSION_REQUEST_CODE);
            } else {
                // Permission already granted, check if we should post test notification
                checkAndPostTestNotification();
            }
        } else {
            // Below Android 13, permission is granted at install time
            checkAndPostTestNotification();
        }
    }

    @Override
    public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[]
grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == PERMISSION_REQUEST_CODE) {
            if (grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                Toast.makeText(this, "Notification permission granted", Toast.LENGTH_SHORT).show();
                checkAndPostTestNotification();
            } else {
                Toast.makeText(this, "Notification permission denied. App may not show modified notifications.",
Toast.LENGTH_LONG).show();
            }
        }
    }

    private void checkAndPostTestNotification() {
        if (!preferenceManager.isTestNotificationSent()) {
            postTestNotification();
            preferenceManager.setTestNotificationSent(true);
        }
    }

    private void postTestNotification() {
        NotificationManager notificationManager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);
        if (notificationManager == null) return;
```

```
        // Create channel if needed (though service also does this)
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel modifiedChannel = new NotificationChannel(
                    MODIFIED_CHANNEL_ID, "Privacy Modifications", NotificationManager.IMPORTANCE_HIGH);
            modifiedChannel.setDescription("Notifications modified by Mirage Notify for privacy");
            notificationManager.createNotificationChannel(modifiedChannel);
        }

        Notification.Builder builder;
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            builder = new Notification.Builder(this, MODIFIED_CHANNEL_ID);
        } else {
            builder = new Notification.Builder(this);
        }

        builder.setContentTitle("Mirage Notify")
                .setContentText("Permission granted!")
                .setSmallIcon(R.drawable.ic_notification)
                .setAutoCancel(true);

        notificationManager.notify(9999, builder.build());
    }

    private boolean onNavigationItemSelected(@NonNull MenuItem item) {
        Fragment fragment = null;
        int itemId = item.getItemId();

        if (itemId == R.id.nav_home) {
            fragment = new HomeFragment();
            setTitle(R.string.home_title);
        } else if (itemId == R.id.nav_rules) {
            fragment = new RulesFragment();
            setTitle(R.string.rules_title);
        } else if (itemId == R.id.nav_logs) {
            fragment = new LogsFragment();
            setTitle(R.string.logs_title);
        } else if (itemId == R.id.nav_settings) {
            fragment = new SettingsFragment();
            setTitle(R.string.settings_title);
        }

        return fragment != null && loadFragment(fragment);
    }

    private boolean loadFragment(Fragment fragment) {
        if (fragment != null) {
            getSupportFragmentManager()
                    .beginTransaction()
                    .replace(R.id.fragment_container, fragment)
                    .commit();
            return true;
        }
        return false;
    }
}
```

Activity_main.xml

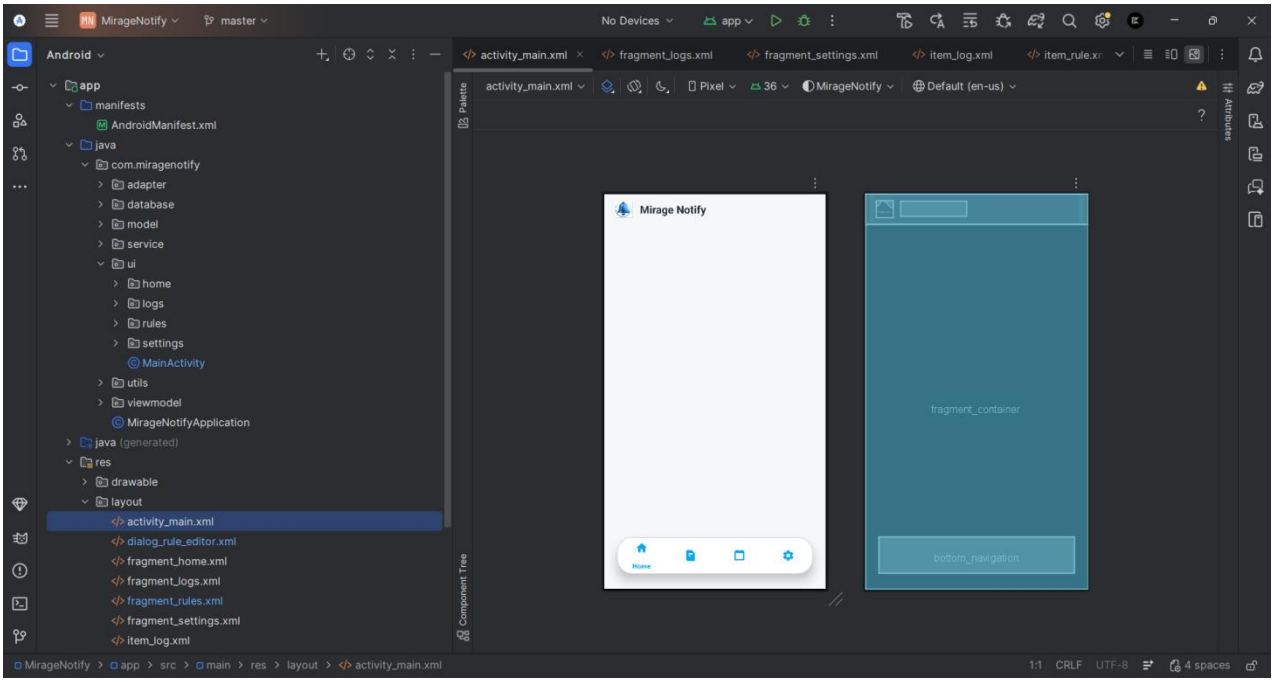
```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/background"
    android:clipChildren="false"
    android:clipToPadding="false">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:background="@color/background"
        app:elevation="0dp">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:contentInsetStart="0dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="match_parent"
                android:gravity="center_vertical"
                android:orientation="horizontal"
                android:paddingStart="20dp"
                android:paddingEnd="20dp">

                <ImageView
                    android:layout_width="32dp"
                    android:layout_height="32dp"
                    android:src="@mipmap/ic_launcher"
                    android:elevation="2dp" />

                <TextView
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginStart="14dp"
                    android:text="@string/app_name"
                    android:textColor="@color/text_primary"
                    android:textSize="20sp"
                    android:textStyle="bold" />

            </LinearLayout>

        </androidx.appcompat.widget.Toolbar>

    </com.google.android.material.appbar.AppBarLayout>

    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:paddingBottom="100dp"
        android:clipToPadding="false"
        app:layout_behavior="@string/appbar_scrolling_view_behavior" />
```

```
<!-- Enhanced Floating Bottom Navigation with Selected Label Visibility -->
<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom"
    android:layout_marginStart="24dp"
    android:layout_marginEnd="24dp"
    android:layout_marginBottom="28dp"
    app:cardCornerRadius="28dp"
    app:cardElevation="10dp"
    app:cardBackgroundColor="@color/surface"
    app:strokeWidth="1dp"
    app:strokeColor="@color/border"
    android:clipChildren="false"
    android:clipToPadding="false">

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation"
        android:layout_width="match_parent"
        android:layout_height="68dp"
        android:background="@android:color/transparent"
        app:elevation="0dp"
        app:itemIconSize="24dp"
        app:itemIconTint="@color/primary"
        app:itemTextColor="@color/primary"
        app:itemRippleColor="@color/primary_light"
        app:labelVisibilityMode="selected"
        app:menu="@menu/bottom_nav_menu" />

</com.google.android.material.card.MaterialCardView>

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

APPENDIX

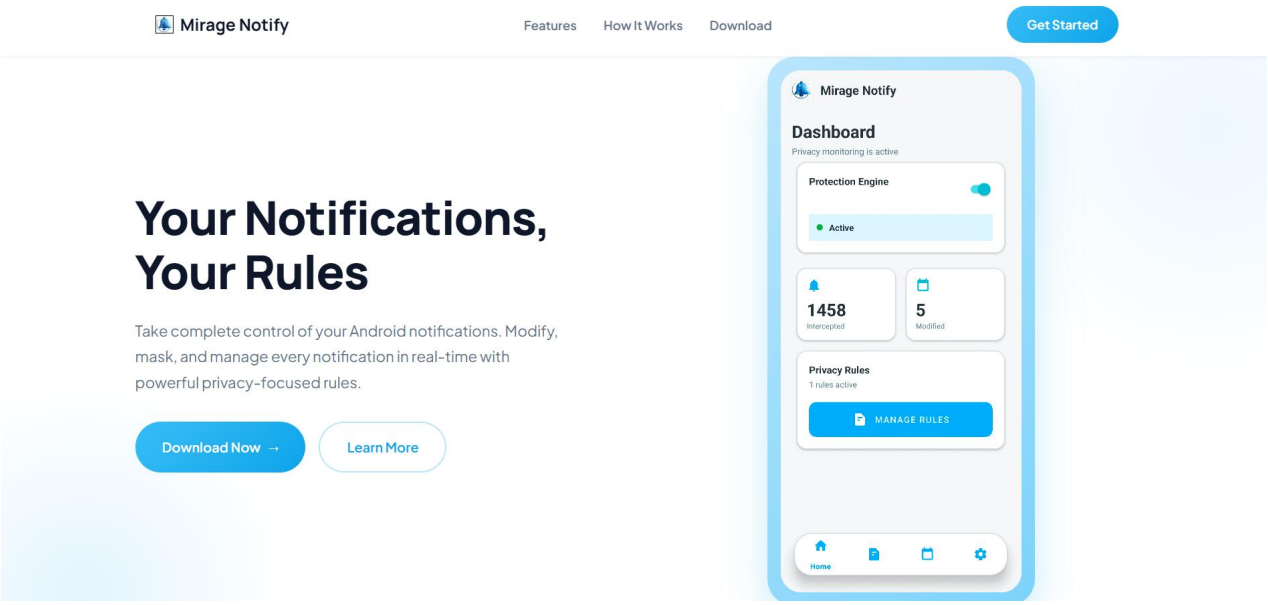
APPENDIX-1

Android Studio



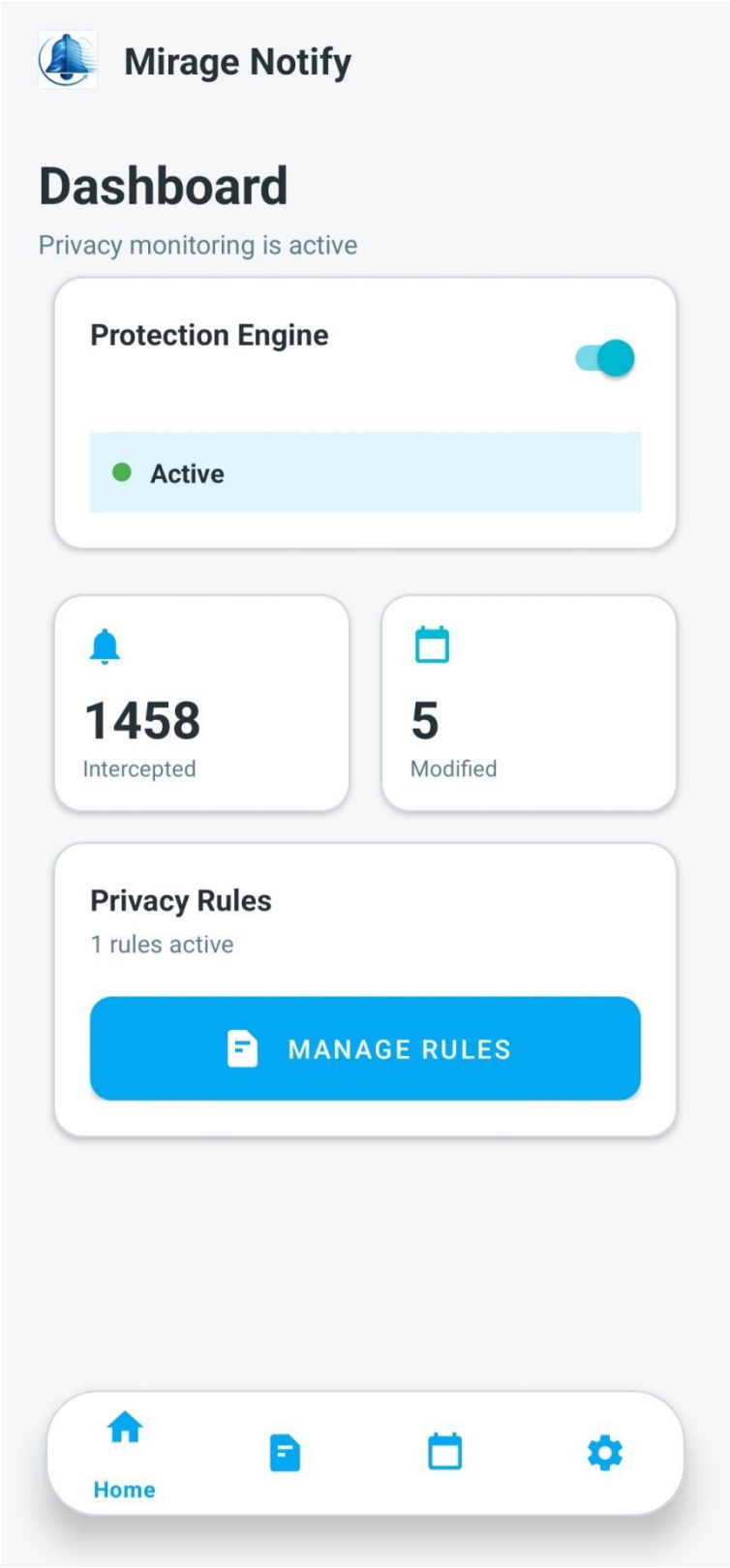
APPENDIX-2

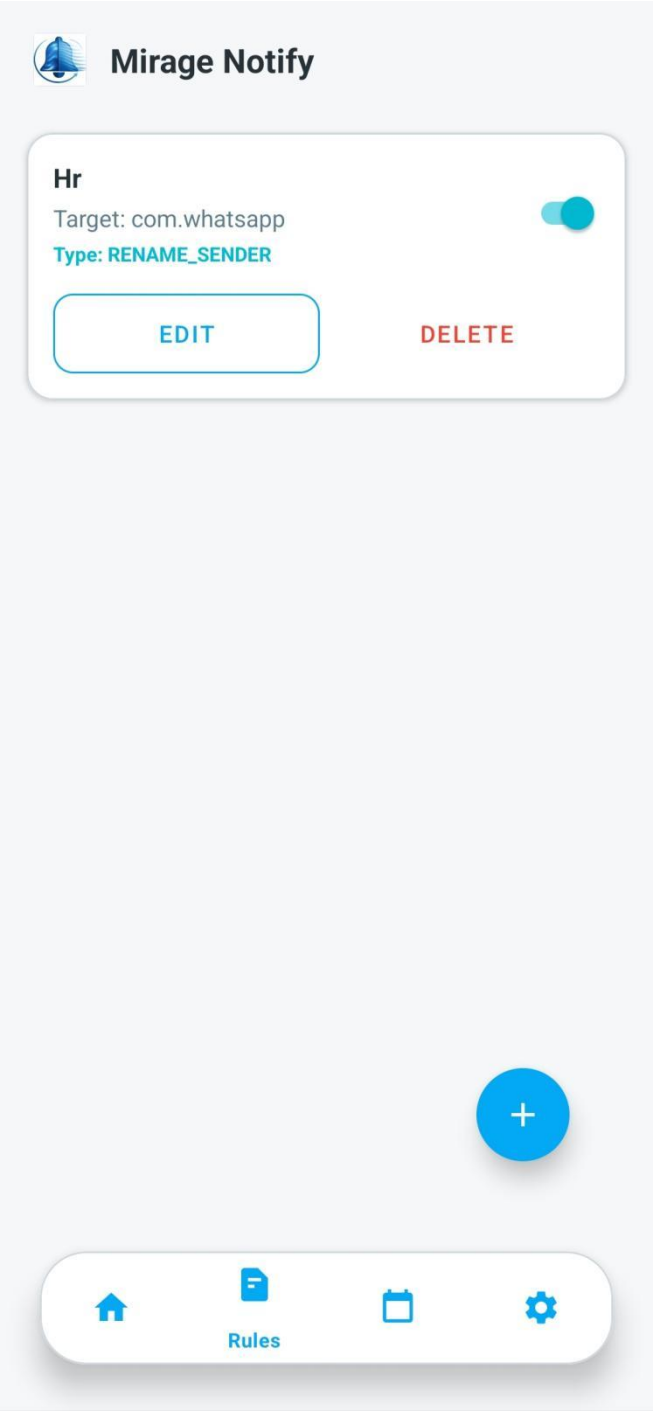
Website

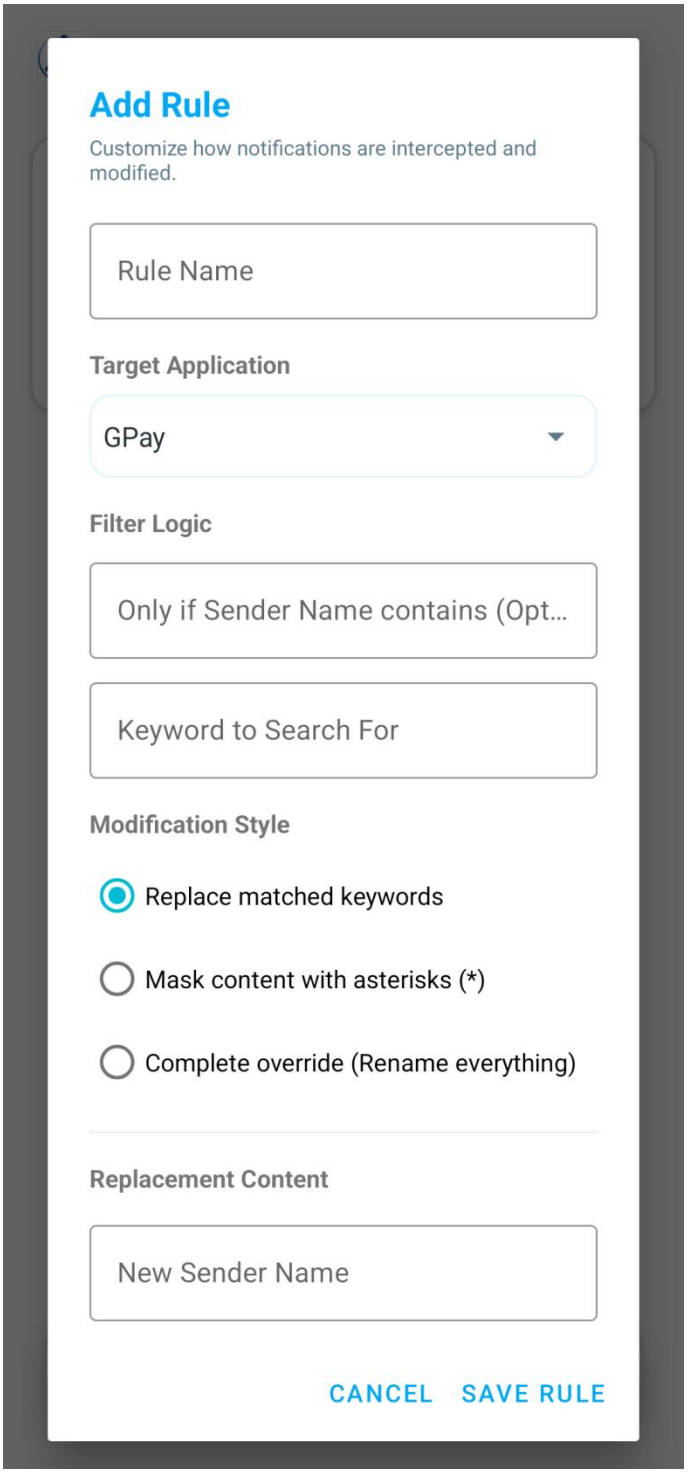


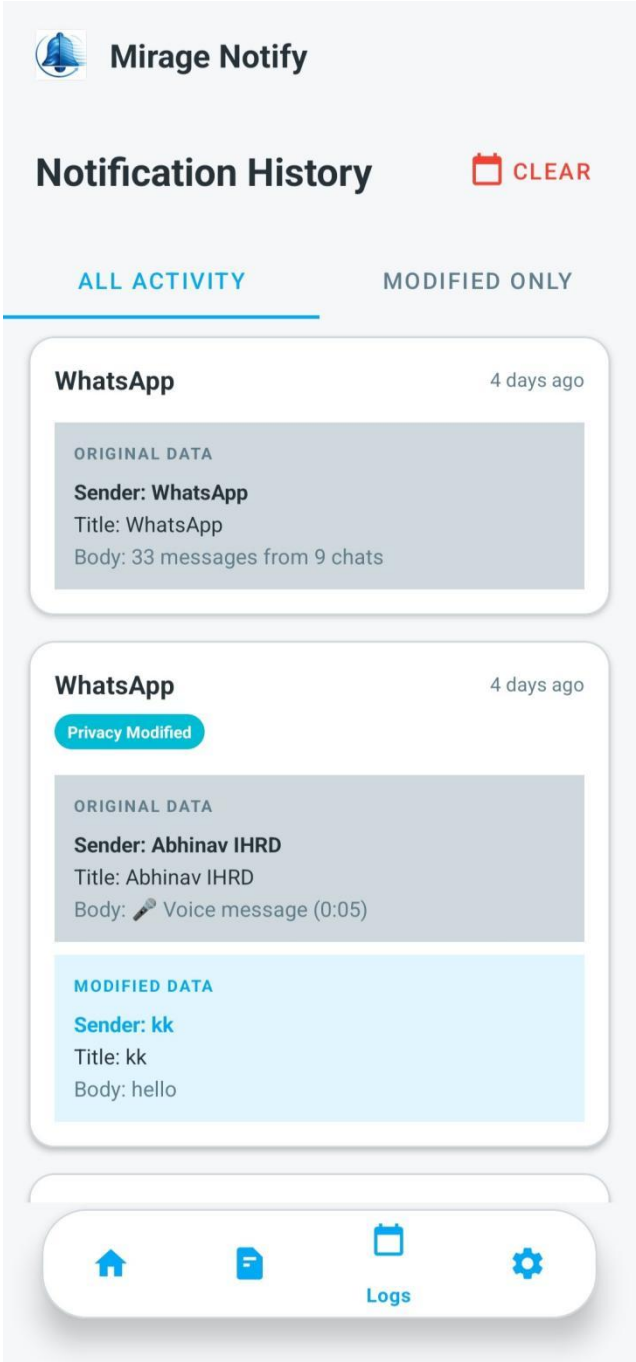
APPENDIX-3

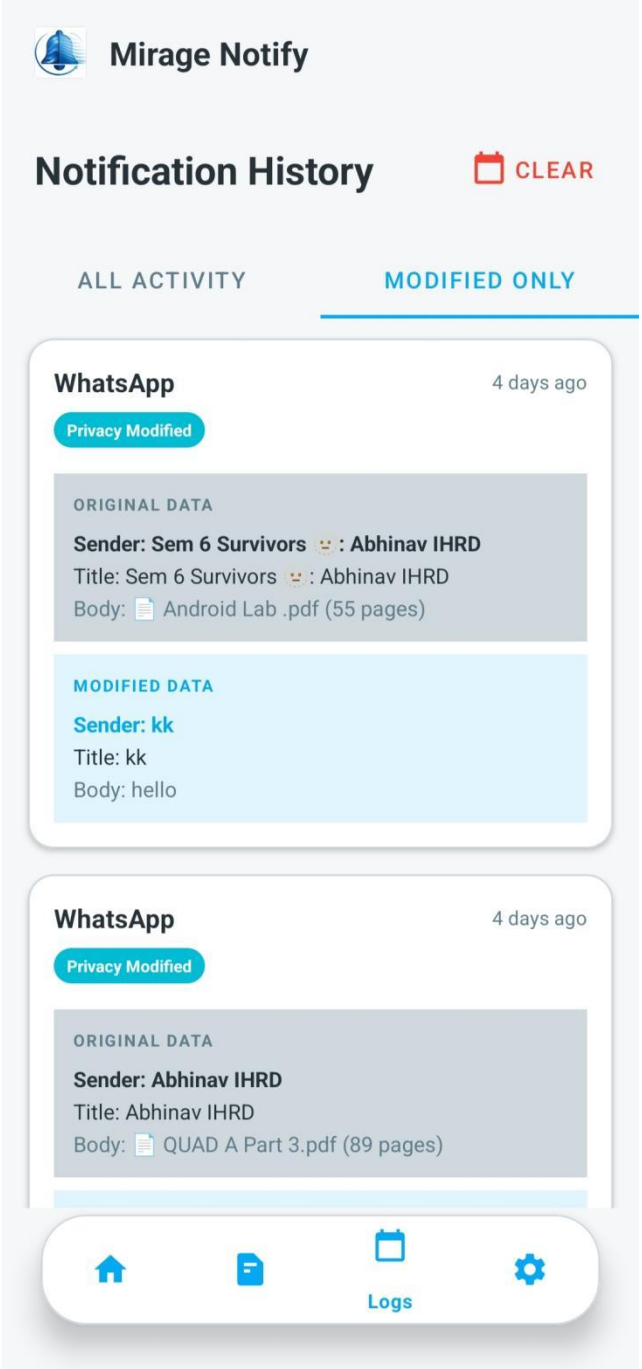
Android Application

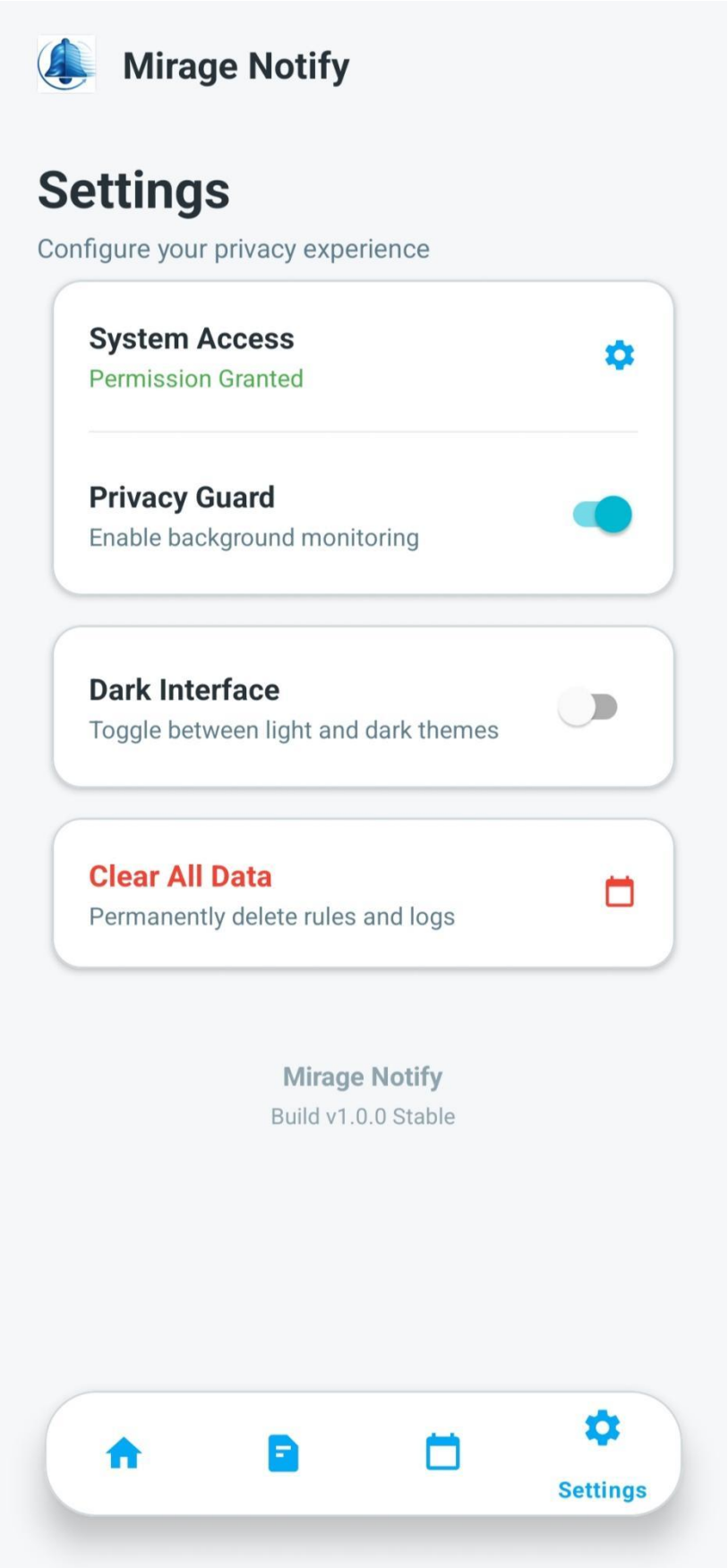



Mirage Notify
Hr
Target: com.whatsapp
Type: RENAME_SENDER
EDIT
DELETE
Rules


Add Rule
Customize how notifications are intercepted and modified.
Rule Name
Target Application
GPay
Filter Logic
Only if Sender Name contains (Opt...
Keyword to Search For
Modification Style
Replace matched keywords
Mask content with asterisks (*)
Complete override (Rename everything)
Replacement Content
New Sender Name
CANCEL
SAVE RULE


Mirage Notify
Notification History
CLEAR
ALL ACTIVITY
MODIFIED ONLY
WhatsApp
4 days ago
ORIGINAL DATA
Sender: WhatsApp
Title: WhatsApp
Body: 33 messages from 9 chats
WhatsApp
4 days ago
Privacy Modified
ORIGINAL DATA
Sender: Abhinav IHRD
Title: Abhinav IHRD
Body: Voice message (0:05)
MODIFIED DATA
Sender: kk
Title: kk
Body: hello
Logs


Mirage Notify
Notification History
CLEAR
ALL ACTIVITY
MODIFIED ONLY
WhatsApp
4 days ago
Privacy Modified
ORIGINAL DATA
Sender: Sem 6 Survivors : Abhinav IHRD
Title: Sem 6 Survivors : Abhinav IHRD
Body: Android Lab .pdf (55 pages)
MODIFIED DATA
Sender: kk
Title: kk
Body: hello
WhatsApp
4 days ago
Privacy Modified
ORIGINAL DATA
Sender: Abhinav IHRD
Title: Abhinav IHRD
Body: QUAD A Part 3.pdf (89 pages)
Logs


Mirage Notify
Settings
Configure your privacy experience
System Access
Permission Granted
Privacy Guard
Enable background monitoring
Dark Interface
Toggle between light and dark themes
Clear All Data
Permanently delete rules and logs
Mirage Notify
Build v1.0.0 Stable
Settings